

Operating System

Semester III — Detailed Unit-wise Notes (Course Code: 2418302)

Unit 1.0 — Operating System Concepts

1.1 Operating System — Concept, Components, System Software

An **Operating System (OS)** is system software that acts as an interface between the computer hardware and the user/application programs, managing all hardware and software resources.

- bullet **Components of OS:** process management, memory management, file management, device (I/O) management, and security/protection modules.
- bullet **System Software:** programs that manage and control computer hardware so that application software can perform required tasks (e.g., OS, compilers, device drivers) — distinct from application software, which performs user-specific tasks.

1.2 Functions of OS

- bullet **Program/Process Management:** creating, scheduling, suspending, and terminating processes.
- bullet **Resource Management:** allocating CPU, memory, and I/O devices efficiently among competing processes.
- bullet **File Management:** organizing, storing, retrieving, naming, and protecting files.
- bullet **Device Management:** managing I/O devices via drivers, buffering, and scheduling device requests.
- bullet **Security and Protection:** controlling access to system resources, authenticating users, preventing unauthorized access.

1.3 Views of OS

- bullet **User view:** the OS is seen as a means to make the computer convenient to use, maximizing user productivity and ease of use (especially important in single-user systems and GUIs).
- bullet **System view:** the OS is seen as a **resource allocator/manager**, deciding how to allocate CPU, memory, storage, and I/O devices among multiple users/programs, and as a **control program**, preventing errors and improper use.

1.4 Types of Operating Systems and Their Characteristics

- bullet **Batch Operating System:** jobs with similar requirements are grouped ("batched") and executed together without user interaction during execution; reduces idle CPU time but lacks interactivity.
- bullet **Multiprogramming OS:** multiple programs reside in memory simultaneously; the CPU switches between them to maximize utilization, especially during I/O wait times.
- bullet **Time-Shared OS:** extends multiprogramming by giving each process/user a small time slice (time quantum) of CPU time in round-robin fashion, providing quick interactive response to multiple users.
- bullet **Multiprocessing OS:** uses two or more CPUs within a single computer system, allowing true parallel execution of processes and increased throughput.
- bullet **Distributed OS:** manages a group of independent, networked computers and makes them appear as a single coherent system to users, enabling resource sharing across machines.
- bullet **Real-Time OS (RTOS):** designed for applications with strict timing constraints (hard or soft deadlines), e.g., embedded systems, industrial control.

- bullet **Mobile OS:** optimized for handheld/mobile devices, balancing power efficiency, touch interfaces, and limited resources (e.g., Android, iOS).

1.5 Services of Operating System

Common OS services include: program execution, I/O operations, file-system manipulation, communication (between processes, possibly on different systems), error detection, resource allocation, accounting, and protection/security.

1.6 System Calls

A **system call** is the programmatic interface through which a user/application program requests a service from the operating system kernel (e.g., to create a file, allocate memory, or create a process).

bullet **Types of system calls:**

bullet **Process control** (create, terminate, wait, load, execute a process)

bullet **File management** (create, delete, open, close, read, write a file)

bullet **Device management** (request, release, read, write a device)

bullet **Information maintenance** (get/set time, date, system data)

bullet **Communication** (send/receive messages, create/delete communication connections)

Unit 2.0 — Process Management

2.1 Process — Process States, Process Control Block (PCB)

A **process** is a program in execution; it is an active entity with its own memory space, unlike a program which is a passive set of instructions.

bullet **Process states:**

bullet **New** — process is being created.

bullet **Ready** — process is waiting to be assigned to the CPU.

bullet **Running** — instructions are being executed by the CPU.

bullet **Waiting/Blocked** — process is waiting for some event (e.g., I/O completion).

bullet **Terminated** — process has finished execution.

bullet **Process Control Block (PCB)**: a data structure maintained by the OS for every process, storing information such as process ID, process state, program counter, CPU registers, CPU scheduling information, memory-management information, accounting information, and I/O status. The PCB is essential for context switching.

2.2 Process Scheduling

bullet **Scheduling Queues:**

bullet **Job queue** — set of all processes in the system.

bullet **Ready queue** — set of processes residing in main memory, ready to execute.

bullet **Device/I/O queue** — set of processes waiting for a particular I/O device.

bullet **Schedulers:**

bullet **Long-term scheduler** — selects which jobs are brought into the ready queue (controls degree of multiprogramming).

bullet **Short-term (CPU) scheduler** — selects which ready process is executed next by the CPU.

bullet **Medium-term scheduler** — handles swapping processes in/out of main memory.

bullet **Context Switch**: the process of saving the state (PCB) of a currently running process and loading the saved state of the next process to be run; this is pure overhead since no useful work is done during the switch.

2.3 Inter-Process Communication (IPC)

IPC allows processes to communicate and synchronize their actions.

bullet **Shared Memory System**: a region of memory is shared between cooperating processes; processes can read/write to this region directly — fast, but requires synchronization to avoid race conditions.

bullet **Message Passing System**: processes communicate by exchanging messages via the OS (using send()/receive() primitives) — easier to implement in distributed systems and avoids shared-memory synchronization issues, but generally slower than shared memory.

2.4 Threads

A **thread** is the smallest unit of CPU execution within a process; a process can have multiple threads sharing the same address space.

- Benefits:** responsiveness, faster context switching among threads of the same process, resource sharing, economy (cheaper than process creation), scalability on multiprocessor systems.

- User threads:** managed by a user-level thread library without kernel support — fast to create/manage but the kernel only sees one process.

- Kernel threads:** managed directly by the OS kernel — kernel can schedule each thread independently, but management overhead is higher.

- Multithreading Models:**

- Many-to-One:** many user threads are mapped to a single kernel thread; simple but blocking system calls block the entire process, and no true parallelism.

- One-to-One:** each user thread is mapped to a distinct kernel thread; provides more concurrency but creating a kernel thread is more costly (overhead per thread).

- Many-to-Many:** multiplexes many user-level threads to a smaller or equal number of kernel threads — combines the best features of both models, allowing flexibility and true parallelism.

Unit 3.0 — CPU Scheduling and Algorithm

3.1 Scheduling Types

- bullet **Scheduling Objectives:** maximize CPU utilization and throughput; minimize turnaround time, waiting time, and response time; ensure fairness.
- bullet **CPU and I/O burst cycles:** process execution alternates between CPU bursts (computation) and I/O bursts (waiting for I/O); CPU scheduling decisions are made at the end of CPU bursts.
- bullet **Pre-emptive scheduling:** the CPU can be forcibly taken away from a running process (e.g., when a higher-priority process arrives or the time quantum expires).
- bullet **Non-pre-emptive scheduling:** once the CPU is allocated to a process, it keeps the CPU until it voluntarily releases it (terminates or blocks for I/O).
- bullet **Scheduling criteria:** CPU utilization, throughput, turnaround time, waiting time, response time.

3.2 Types of Scheduling Algorithms

- bullet **First Come First Served (FCFS):** processes are executed in the order they arrive (a simple FIFO queue); non-preemptive; simple but can cause the **convoy effect** (short processes wait behind a long one).
- bullet **Shortest Job First (SJF):** the process with the smallest estimated CPU burst time is executed next; can be preemptive or non-preemptive; minimizes average waiting time but requires knowledge of burst time in advance and can cause starvation of long jobs.
- bullet **Shortest Remaining Time First (SRTF):** the preemptive version of SJF — if a new process arrives with a shorter burst time than the remaining time of the current process, the CPU is switched to the new process.
- bullet **Round Robin (RR):** each process is given a fixed small time slice (**time quantum**) in cyclic order; preemptive; provides fair CPU allocation and good response time for interactive systems, but performance depends heavily on quantum size.
- bullet **Priority Scheduling:** each process is assigned a priority; the CPU is allocated to the process with the highest priority; can be preemptive or non-preemptive; may cause **starvation** of low-priority processes (solved using **aging**, gradually increasing priority of waiting processes).
- bullet **Multilevel Queue Scheduling:** the ready queue is divided into several separate queues based on process characteristics (e.g., foreground/interactive vs background/batch), each queue having its own scheduling algorithm, with scheduling also required between queues.
- bullet **Turnaround Time and Average Waiting Time calculation:**
- bullet $\text{Turnaround Time} = \text{Completion Time} - \text{Arrival Time}$
- bullet $\text{Waiting Time} = \text{Turnaround Time} - \text{Burst Time}$
- bullet Average values are computed by summing individual times and dividing by the number of processes.

3.3 Deadlock

A **deadlock** is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource held by another process in the set, so none can proceed.

- bullet **System Models:** resources are represented as a **resource-allocation graph**, with processes and resources as nodes, and request/assignment edges between them.

bullet **Necessary Conditions Leading to Deadlock** (all four must hold simultaneously):

1. **Mutual Exclusion** — only one process can use a resource at a time.
2. **Hold and Wait** — a process holding at least one resource is waiting to acquire additional resources held by others.
3. **No Preemption** — resources cannot be forcibly taken away from a process; they must be released voluntarily.
4. **Circular Wait** — a set of processes are waiting for each other in a circular chain.

bullet **Deadlock Handling:**

bullet **Prevention:** design the system so that at least one of the four necessary conditions can never hold (e.g., requesting all resources at once, imposing a resource ordering).

bullet **Avoidance:** the OS dynamically examines resource-allocation requests and grants only those that keep the system in a "safe state" (e.g., **Banker's Algorithm**).

Unit 4.0 — Memory Management

4.1 Basic Memory Management

- bullet **Partitioning**: dividing main memory into sections to hold processes.
- bullet **Fixed Partitioning**: memory divided into fixed-size partitions at system startup; simple but causes internal fragmentation.
- bullet **Variable (Dynamic) Partitioning**: partitions are created dynamically to exactly fit the size of incoming processes; avoids internal fragmentation but causes external fragmentation.
- bullet **Free Space Management Techniques**:
- bullet **Bitmap (Bit vector)**: each block of memory is represented by one bit (1 = allocated, 0 = free); simple but requires scanning to find free blocks.
- bullet **Linked List**: free blocks are linked together in a list, each node storing the starting address and size of a free segment.

4.2 Virtual Memory

Virtual memory is a technique that allows execution of processes that may not be completely in main memory, giving each process the illusion of a large, contiguous address space.

- bullet **Paging**: divides logical memory into fixed-size blocks called **pages** and physical memory into equal-sized **frames**; pages are mapped to frames using a **page table**, eliminating external fragmentation.
- bullet **Segmentation**: divides a program into variable-sized logical units called **segments** (e.g., code segment, data segment, stack segment) based on the program's logical structure.
- bullet **Fragmentation**:
- bullet **Internal fragmentation** — wasted memory inside an allocated block (block larger than needed).
- bullet **External fragmentation** — free memory is scattered in small non-contiguous chunks, insufficient to satisfy a request even though total free memory is enough.
- bullet **Page Fault**: occurs when a process accesses a page that is not currently loaded in main memory; the OS must fetch it from secondary storage (disk), causing a performance penalty.

4.3 Page Replacement Algorithms

When a page fault occurs and memory is full, the OS must choose an existing page to remove ("replace") to make room — this decision is made by a page replacement algorithm.

- bullet **FIFO (First In First Out)**: replaces the page that has been in memory the longest, regardless of how often it's used; simple but can suffer from **Belady's anomaly** (more frames can lead to more page faults).
- bullet **LRU (Least Recently Used)**: replaces the page that has not been used for the longest time, based on the assumption that recently used pages are likely to be used again soon; performs well but requires tracking usage history (overhead).
- bullet **Optimal (OPT)**: replaces the page that will not be used for the longest time in the future; gives the lowest possible page-fault rate but is impossible to implement in practice (requires future knowledge) — used mainly as a theoretical benchmark.
- bullet **Page fault calculation**: for a given page-reference string and number of frames, the algorithm is simulated step by step, counting a "fault" each time a page must be loaded that is not already present.

Unit 5.0 — File Management

1.1 File — Concepts, Attributes, Operations, Types, File System Structure

A **file** is a named collection of related information recorded on secondary storage.

- Attributes:** name, identifier, type, size, location, protection, time/date, and user identification.
- Operations:** create, write, read, reposition (seek), delete, truncate, open, close.
- Types of files:** regular files (text/binary), directory files, special device files.
- File System Structure:** organizes how data is stored, named, and retrieved — typically a layered structure consisting of the application programs, logical file system, file-organization module, basic file system, I/O control, and the physical devices.

1.2 Access Methods, File Allocation Methods

- Access Methods:**
- Sequential Access:** data is accessed in order, one record after another, from the beginning — simple, matches tape storage.
- Direct (Random) Access:** a file is made up of fixed-length logical records that can be read/written in any order by specifying the relative block/record number.
- Swapping:** the technique of moving entire processes between main memory and secondary storage (swap space) to free up memory for other processes.
- File Allocation Methods** (how disk blocks are allocated to files):
- Contiguous Allocation:** each file occupies a set of contiguous blocks on disk; fast sequential and direct access, but suffers from external fragmentation and difficulty growing files.
- Linked Allocation:** each file is a linked list of disk blocks scattered anywhere on disk; no external fragmentation and files can grow easily, but slow direct access and overhead of pointers.
- Indexed Allocation:** brings all pointers for a file's blocks together into one index block; supports direct access without external fragmentation, but requires extra space for the index block.

1.3 Directory Structure, Disk Organization

- Directory Structure:**
- Single-level directory:** all files are contained in one directory — simple but causes naming conflicts and lacks grouping.
- Two-level directory:** each user has a separate directory — solves naming conflicts between users but limits grouping within a user's files.
- Tree-structured directory:** directories can contain subdirectories, forming a hierarchical tree — allows flexible grouping and efficient searching, used in most modern OS.
- Disk Organization and Disk Structure:**
- Physical structure:** the actual hardware layout of a disk — platters, tracks, sectors, cylinders, and read/write heads.
- Logical structure:** how the OS views and organizes data on the disk independent of the physical layout — e.g., as a sequential array of logical blocks, abstracted from the actual physical addressing.